
Modello fisico del banco e training della policy SAC

Appunti di ingegneria

Progetto fan-sac-controller (controllo in forza di un fan con Reinforcement Learning)
Autore Matteo Faggian
Periodo settembre – ottobre 2026
Codice `controller/env_fan.py` (versione F), `controller/sysid_params.json`

Come leggere questi appunti Descrivono il simulatore usato per addestrare la policy (il modello fisico del banco, costruito sui dati misurati), la formulazione del problema di controllo come problema di Reinforcement Learning, la teoria dell'algoritmo SAC e la sequenza di esperimenti che ha portato alla policy F. Il codice riportato non è ricopiato: è incluso direttamente da `controller/env_fan.py`, con i numeri di riga del file. I numeri sperimentali provengono dai file del repository (`data/`, `models/`) e dalle valutazioni descritte nel testo.

Indice

1	Il sistema	2
1.1	Descrizione	2
1.2	Equilibrio del carrello	2
1.3	Dimensionamento del compito	2
2	System identification: i dati	2
3	Modello del rotore	4
3.1	Perché un primo ordine	4
3.2	Discretizzazione esatta	4
3.3	Deadband con isteresi e ritardo di avvio	4
3.4	Ritardo di attuazione	5
4	Modello della misura e del disturbo	5
4.1	Misura	5
4.2	Disturbo: inclinazione della corsia	6
4.3	Validazione del simulatore	6
5	Domain randomization	7
6	Formulazione come problema di RL	8
6.1	Il ciclo	8
6.2	Azione	9
6.3	Osservazione	9
6.4	Ricompensa	10
7	L'algoritmo SAC	10
7.1	Obiettivo	10
7.2	Critic: la Q "soft"	10
7.3	Actor e trucco della riparametrizzazione	11
7.4	Temperatura automatica	11
7.5	Off-policy e replay buffer	11
7.6	Configurazione usata	11
8	Esperimenti	12
8.1	Metriche	12
8.2	Risultati	12
8.3	Sul banco: la policy E	13
8.4	Esperimento F: robustezza al ritardo	14

9	Riepilogo delle lezioni	14
A	Codice: parametri e costruttore dell'ambiente	15
B	Codice: reset dell'episodio	16

1 Il sistema

1.1 Descrizione

Un carrello scorre su una corsia la cui inclinazione θ è cambiata a mano. Sul carrello è montato un fan (motore brushless con elica, pilotato da un ESC). Alla fine della corsia una *load cell* letta da un HX711 misura la forza con cui il carrello la spinge. Non c'è alcun sensore di inclinazione. L'obiettivo è mantenere costante la forza misurata, $F_{lc} = F_{ref}$, qualunque sia θ .

1.2 Equilibrio del carrello

Si proietta l'equilibrio delle forze lungo l'asse della corsia, positivo verso la cella. Sul carrello agiscono la spinta del fan T , la componente del peso $-mg \sin \theta$ (con $\theta > 0$ quando la cella sta più in alto del carrello) e la reazione della cella $-N$. Il carrello è bloccato contro la cella, quindi l'accelerazione è nulla:

$$T - mg \sin \theta - N = 0 \quad \Rightarrow \quad N = T - mg \sin \theta. \quad (1)$$

La cella misura N più un errore di zero e un rumore:

$$F_{lc} = T - mg \sin \theta + b + n \quad (2)$$

Ipotesi: carrello sempre in contatto con la cella e attriti trascurabili rispetto a T (l'effetto dell'attrito residuo è assorbito dal bias b , che viene randomizzato).

1.3 Dimensionamento del compito

Per tenere $F_{lc} = F_{ref}$ il fan deve dare $T = F_{ref} + mg \sin \theta$. Con $\theta \in [-\theta_{max}, \theta_{max}]$ la spinta richiesta varia in $[F_{ref} - mg \sin \theta_{max}, F_{ref} + mg \sin \theta_{max}]$. Questo intervallo deve stare nella zona in cui il motore lavora bene, ricavata dalla system identification: sopra ≈ 0.4 N (lontano dalla soglia di arresto) e sotto ≈ 1.9 N (margine rispetto alla saturazione a ≈ 2.2 N imposta dall'alimentatore):

$$F_{ref} = \frac{0.4 + 1.9}{2} = 1.15 \text{ N}, \quad \sin \theta_{max} \leq \frac{0.75}{0.317 \cdot 9.81} = 0.241 \Rightarrow \theta_{max} \approx 14^\circ. \quad (3)$$

2 System identification: i dati

Il modello non usa parametri ipotizzati: sono tutti misurati sul banco con il test descritto in docs/03_system_identification.md (scalini lenti per la mappa statica, gradini per la dinamica, partenze da fermo, tratti a motore fermo). I risultati sono salvati in controller/sysid_params.json.

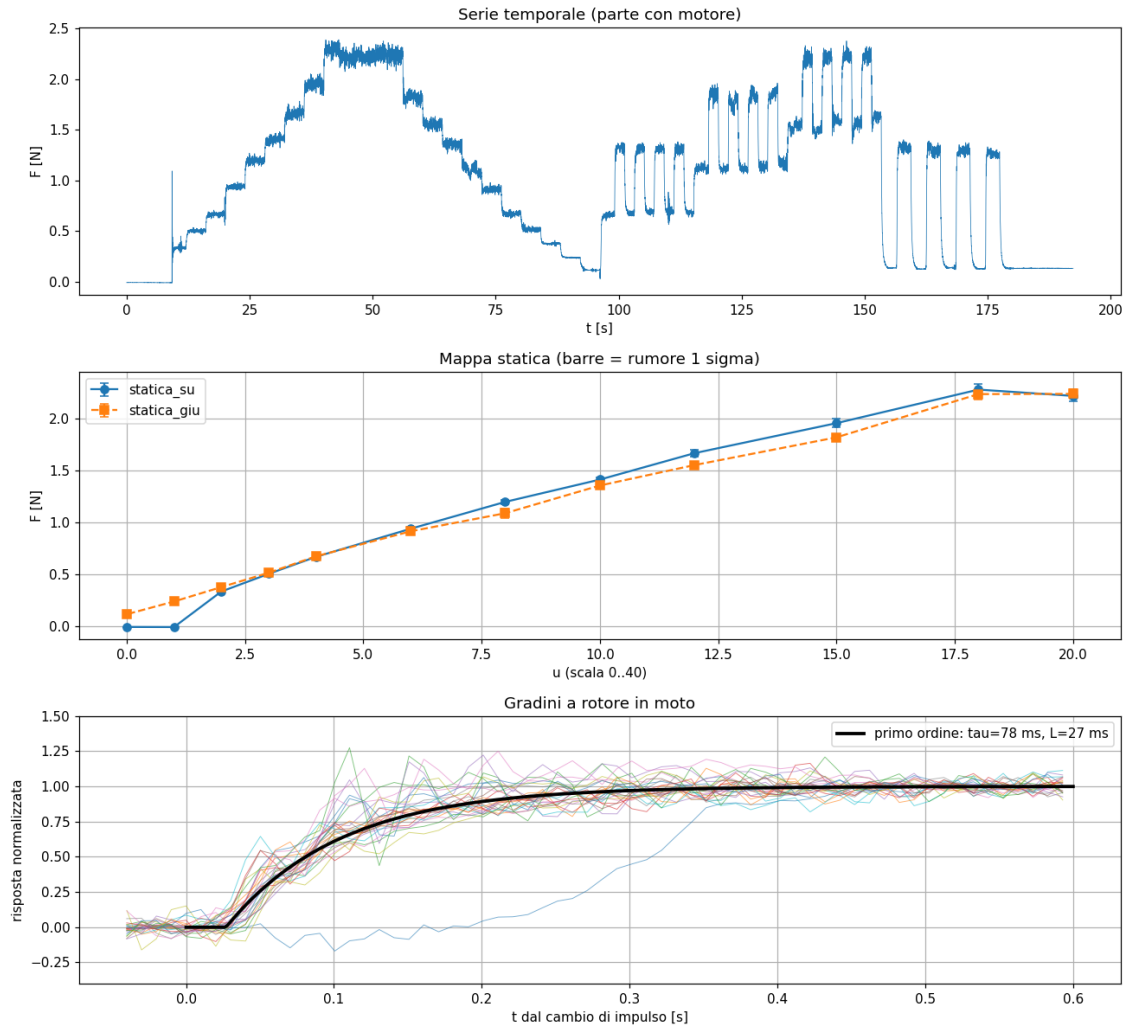


Figura 1: Test di identificazione: serie temporale, mappa statica (salita e discesa) e gradini a rotore in moto normalizzati, con il modello del primo ordine sovrapposto.

Grandezza	Simbolo	Valore misurato
Costante di tempo, rotore in moto	τ_{on}	78 ms (33–112 ms tra le ripetizioni)
Costante di tempo, spegnimento	τ_{off}	174 ms
Ritardo puro, rotore in moto	L	27 ms
Ritardo di avvio da fermo	L_{avvio}	200 ms
Soglia di avvio / arresto	$u_{avvio}, u_{arresto}$	1.5 / 0.5 (scala 0–40)
Rumore	σ	1.1 mN + 2.08% · T
Spostamento dello zero dopo i test	b	+0.132 N
Spinta massima		≈ 2.21 N (saturazione)

Mappa statica “in moto” (zero sottratto; media di salita e discesa dove il motore gira in entrambe):

u	1	2	3	4	6	8	10	12	15	18	20
F [N]	0.123	0.302	0.456	0.617	0.873	1.090	1.331	1.557	1.835	2.206	2.206

La mappa è quasi lineare, circa 0.11 N per unità di u , leggermente concava, e satura a 18.

3 Modello del rotore

3.1 Perché un primo ordine

Il rotore è un'inerzia J accelerata dalla coppia del motore Q_m e frenata dalla coppia aerodinamica $Q_a(\omega)$:

$$J\dot{\omega} = Q_m(u, \omega) - Q_a(\omega). \quad (4)$$

L'ESC regola la velocità in modo che, a regime, ω dipenda solo dal comando u . Linearizzando attorno a un punto di lavoro, la deviazione $\delta\omega$ obbedisce a $J\delta\dot{\omega} = -k_\omega\delta\omega + k_u\delta u$, cioè a un sistema del primo ordine con costante di tempo $\tau = J/k_\omega$. Poiché la spinta è una funzione regolare di ω ($T \propto \omega^2$), anche le piccole variazioni di spinta seguono, in prima approssimazione, un primo ordine.

Questa è la giustificazione del modello; la sua validità è stata verificata sui dati: le risposte ai gradini si sovrappongono bene alla curva di un primo ordine con ritardo (figura precedente). La dispersione di τ (33–112 ms) indica che la linearizzazione vale solo localmente: per questo τ viene randomizzata in training.

3.2 Discretizzazione esatta

Con spinta di regime $T^*(u)$ (dalla mappa statica), il modello continuo è

$$\dot{T} = \frac{T^* - T}{\tau}. \quad (5)$$

Il controllore agisce a passi di $\Delta t = 20$ ms e mantiene il comando costante tra un passo e l'altro (*zero-order hold*). Su un intervallo con T^* costante la soluzione esatta è

$$T(t_k + \Delta t) = T^* + (T(t_k) - T^*)e^{-\Delta t/\tau} \Rightarrow \boxed{T_{k+1} = T_k + \alpha(T^* - T_k), \quad \alpha = 1 - e^{-\Delta t/\tau}} \quad (6)$$

Con $\tau = 78$ ms: $\alpha = 1 - e^{-0.02/0.078} = 0.226$.

Perché non Eulero Il metodo di Eulero darebbe $\alpha_E = \Delta t/\tau = 0.256$, un errore del 13%. Per la τ più piccola randomizzata (≈ 40 ms) l'errore sale al 27%, e per $\tau < \Delta t/2 = 10$ ms Eulero diventerebbe addirittura instabile ($\alpha_E > 2$). La formula esatta è corretta per ogni $\tau > 0$.

3.3 Deadband con isteresi e ritardo di avvio

Dai dati: da fermo il motore parte solo se $u \geq u_{avvio}$, e impiega circa 200 ms a sincronizzarsi prima di spingere; una volta in moto si ferma solo se $u < u_{arresto} < u_{avvio}$. Si modella come una macchina a due stati:

Stato	Condizione	Evoluzione
fermo	$u \geq u_{avvio}$ per $\text{round}(L_{avvio}/\Delta t) = 10$ passi consecutivi	$\rightarrow$ in moto
fermo	altrimenti	T decade con τ_{off} verso 0
in moto	$u < u_{arresto}$	$\rightarrow$ fermo
in moto	altrimenti	T tende a $g \cdot T^*(u)$ con τ_{on}

Il guadagno g (nominale 1) moltiplica la mappa statica; $\tau_{off} > \tau_{on}$ perché in spegnimento il rotore rallenta solo per attrito e resistenza dell'aria, senza frenata attiva.

```

186 def _rotor_step(self, u):
187     """Aggiorna la spinta per un passo, dato il comando u (scala 0..40) che arriva all'ESC.
188     """
189     if not self.in_moto:
190         # motore fermo: parte solo se u resta sopra la soglia per tutto il ritardo di avvio
191         if u >= self.u_avvio:
192             self.conta_avvio += 1
193             if self.conta_avvio >= self.passi_avvio:
194                 self.in_moto = True
195         else:
196             self.conta_avvio = 0
197     elif u < self.u_arresto:
198         # in moto ma sotto la soglia di arresto: il motore si spegne
199         self.in_moto = False
200         self.conta_avvio = 0
201
202     if self.in_moto:
203         target = self.gain * np.interp(u, self.u_tab, self.F_tab) # spinta a regime da
204         # tabella
205         self.thrust += self.alpha_on * (target - self.thrust)
206     else:
207         self.thrust += self.alpha_off * (0.0 - self.thrust) # rallenta per inerzia

```

La funzione `np.interp` interpola linearmente la tabella e, oltre l'ultimo punto, restituisce il valore di bordo: la saturazione a 2.21 N è quindi inclusa automaticamente.

3.4 Ritardo di attuazione

Il comando calcolato al passo k raggiunge il motore dopo n passi: si usa una coda FIFO inizializzata con n zeri. A ogni passo si accoda il nuovo comando e si estrae il più vecchio:

$$u_k^{\text{applicato}} = u_{k-n}. \quad (7)$$

```

276 # ritardo di attuazione: il comando di adesso arriva all'ESC dopo n_ritardo passi
277 self.coda_u.append(u)
278 u_applicato = self.coda_u.popleft()

```

Il ritardo misurato (27 ms) corrisponde a 1–2 passi; sul banco con PC + Arduino il comportamento in anello chiuso è risultato equivalente a 3 passi (sezione 8). In training n è estratto in $\{1, 2, 3, 4\}$; il valore nominale è 3.

4 Modello della misura e del disturbo

4.1 Misura

Implementa l'equazione (2). Il rumore è gaussiano con deviazione standard che cresce con la spinta, perché a motore acceso domina la vibrazione meccanica:

$$n \sim \mathcal{N}(0, \sigma^2), \quad \sigma = s_0 + s_1 |T|, \quad s_0 = 1.1 \text{ mN}, \quad s_1 = 0.0208. \quad (8)$$

A $T = 1.15 \text{ N}$: $\sigma \approx 25 \text{ mN}$.

```

207 def _measure(self):
208     """Lettura della load cell: spinta - gravita' + zero spostato + rumore proporzionale."""
209
210     gravity_force = self.mass * self.g * np.sin(np.radians(self.angle))
211     sigma = self.rumore_s0 + self.rumore_s1 * abs(self.thrust)
212     noise = self.np_random.normal(0.0, sigma)
213     return self.thrust - gravity_force + self.bias + noise

```

4.2 Disturbo: inclinazione della corsia

L'inclinazione non è osservata dalla rete. Ogni 100–300 passi (2–6 s) viene estratto un nuovo angolo in $[-14^\circ, 14^\circ]$, raggiunto con un gradino (probabilità 0.5) o con una rampa di durata 0.5–3 s.

```

163 def _schedule_disturbance(self):
164     """Programma il prossimo cambio di inclinazione (gradino o rampa)."""
165     r = self.np_random
166     self.angle_target = r.uniform(-self.max_angle, self.max_angle)
167     self.steps_to_next_change = int(r.integers(100, 300))
168     if r.random() < 0.5:
169         self.angle_rate = np.inf # gradino
170     else:
171         ramp_time = r.uniform(0.5, 3.0) # [s] rampa
172         self.angle_rate = abs(self.angle_target - self.angle) / ramp_time
173
174 def _update_disturbance(self):
175     self.steps_to_next_change -= 1
176     if self.steps_to_next_change <= 0:
177         self._schedule_disturbance()
178     diff = self.angle_target - self.angle
179     max_step = self.angle_rate * self.dt
180     self.angle += np.clip(diff, -max_step, max_step)

```

4.3 Validazione del simulatore

La stessa sequenza di comandi del test di identificazione è stata data al simulatore (parametri nominali) e confrontata con la misura: RMS della differenza 0.083 N, che scende a 0.069 N togliendo lo spostamento medio dello zero (0.047 N), effetto coperto in training dalla randomizzazione di b .

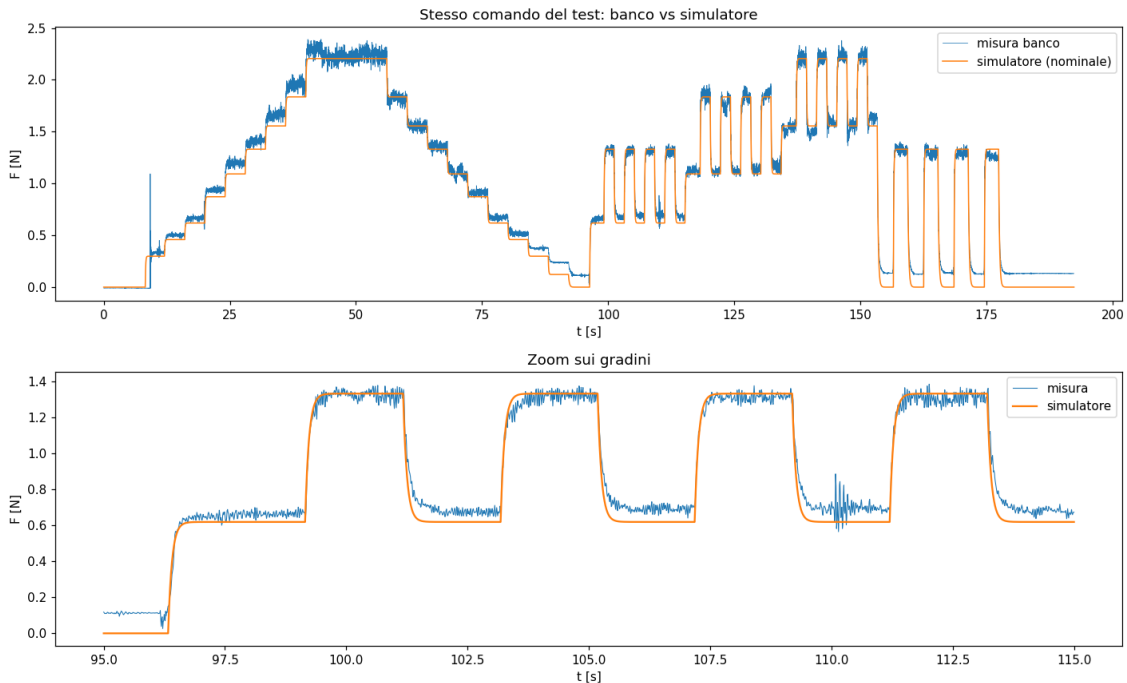


Figura 2: Stesso comando dato al banco e al simulatore. In basso, ingrandimento sui gradini.

5 Domain randomization

Il simulatore non sarà mai identico al banco. Invece di un solo modello si addestra la policy su una famiglia di modelli: a ogni episodio i parametri vengono estratti in intervalli attorno ai valori misurati. Una policy che funziona su tutta la famiglia ha buone probabilità di funzionare anche sul banco, purché il banco “stia dentro” la famiglia.

Parametro	Intervallo	Motivazione
massa m	$\times [0.95, 1.05]$	pesata: incertezza piccola
guadagno g	$[0.85, 1.15]$	isteresi salita/discesa, riscaldamento, incertezza di calibrazione della cella
τ_{on}	$\approx [40, 112]$ ms	dispersione misurata tra le ripetizioni
τ_{off}	$\times [0.85, 1.15]$	
u_{avvio}	$\times [0.8, 1.3]$	
$u_{arresto}$	$\times [0.6, 1.6], \leq u_{avvio}$	l'isteresi non può invertirsi
L_{avvio}	$\times [0.75, 1.25]$	
ritardo n	$\{1, 2, 3, 4\}$ passi	20–80 ms: contiene i ≈ 60 ms visti sul banco con margine
rumore s_1	$\times [0.7, 1.5]$	
bias b	$[-0.05, 0.18]$ N	lo zero misurato si sposta fino a $+0.13$ N

```

127 def _sample_parameters(self):
128     """Parametri fisici dell'episodio: randomizzati attorno ai valori misurati, o nominali.
129     """
129     if self.domain_randomization:
130         r = self.np_random
131         self.mass = self.mass_nom * r.uniform(0.95, 1.05)          # massa pesata: incertezza
132         # guadagno: copre isteresi salita/discesa, riscaldamento e incertezza di
133         # calibrazione
134         self.gain = r.uniform(0.85, 1.15)
135         self.tau = r.uniform(0.8 * self.tau_min + 0.2 * self.tau_nom, self.tau_max) #
136         ~40..112 ms
137         self.tau_off = self.tau_off_nom * r.uniform(0.85, 1.15)
138         self.u_avvio = self.u_avvio_nom * r.uniform(0.8, 1.3)
139         self.u_arresto = min(self.u_arresto_nom * r.uniform(0.6, 1.6), self.u_avvio)
140         self.rit_avvio = self.rit_avvio_nom * r.uniform(0.75, 1.25)
141         self.n_ritardo = int(r.integers(1, 5))                    # 1..4 passi (20-80 ms)
142         self.rumore_s1 = self.rumore_s1_nom * r.uniform(0.7, 1.5)
143         self.bias = r.uniform(-0.05, self.zero_dopo + 0.05)      # zero che si sposta (
144         misurato +0.13 N)
145     else:
146         self.mass = self.mass_nom
147         self.gain = 1.0
148         self.tau = self.tau_nom
149         self.tau_off = self.tau_off_nom
150         self.u_avvio = self.u_avvio_nom
151         self.u_arresto = self.u_arresto_nom
152         self.rit_avvio = self.rit_avvio_nom
153         self.n_ritardo = 3                                       # misurato sul banco via
154         PC+Arduino
155         self.rumore_s1 = self.rumore_s1_nom
156         self.bias = 0.0
157
158     # coefficienti del primo ordine discretizzato in modo esatto:  $\alpha = 1 - e^{-(dt/\tau)}$ 
159     self.alpha_on = 1.0 - np.exp(-self.dt / self.tau)
160     self.alpha_off = 1.0 - np.exp(-self.dt / self.tau_off)
161     self.passi_avvio = int(round(self.rit_avvio / self.dt))      # ritardo di avvio in passi

```

Lezione dalla policy E La randomizzazione deve contenere la realtà *con margine*. La policy E era stata addestrata con ritardo di 1–2 passi; sul banco il ritardo era circa 3 passi e la policy oscillava a ≈ 3 Hz.

6 Formulazione come problema di RL

6.1 Il ciclo

A ogni passo k ($\Delta t = 20$ ms): l'ambiente fornisce l'osservazione o_k , la policy sceglie l'azione a_k , il simulatore avanza e restituisce la ricompensa r_k . Un episodio dura 1000 passi (20 s) e parte con motore fermo e inclinazione casuale.

```

268     def step(self, action):
269         self.t += 1
270
271         # azione [-1, 1] -> comando normalizzato c [c_min, 1] -> u nella scala del banco
272         a = float(np.clip(action[0], -1.0, 1.0))
273         c = self.c_min + (1.0 - self.c_min) * 0.5 * (a + 1.0)
274         u = c * self.u_max
275
276         # ritardo di attuazione: il comando di adesso arriva all'ESC dopo n_ritardo passi
277         self.coda_u.append(u)
278         u_applicato = self.coda_u.popleft()
279
280         self._update_disturbance()
281         self._rotor_step(u_applicato)
282
283         force = self._measure()
284         self.df_filt += self.beta_df * ((force - self.last_force) - self.df_filt)
285         error = self.force_ref - force
286         self.integral = float(np.clip(self.integral + error * self.dt, -self.i_max, self.i_max))
287
288         # reward normalizzato
289         # Termine quadratico e_n^2: punisce molto gli errori grandi, ma vicino a zero e' quasi
290         # piatto (derivata 2*e_n -> 0): un errore costante di 0.15 N costa solo 0.017 a passo,
291         # e la policy SAC v2 si "accontentava" di lasciarlo. Il termine lineare |e_n| ha
292         derivata
293         # costante: anche l'ultimo pezzetto di errore viene pagato, e la policy e' spinta ad
294         annullarlo.
295         e_n = error / self.force_ref
296         du = c - self.last_c
297         reward = -(e_n ** 2) - self.w_e1 * abs(e_n)
298                 - self.w_du * du ** 2 - self.w_u * c ** 2)
299
300         self.last_force = force
301         self.registra_comando(c)
302         obs = self._get_obs(force, self.df_filt, error)
303
304         terminated = False
305         truncated = self.t >= self.max_episode_steps
306         info = {
307             "force": force,
308             "force_ref": self.force_ref,
309             "angle": self.angle,
310             "ESC": c,                # comando normalizzato 0..1 (come nella v1)
311             "u": u,                  # comando nella scala del banco 0..40
312             "thrust": self.thrust,
313             "in_moto": self.in_moto,
314         }

```

```
313 return obs, float(reward), terminated, truncated, info
```

6.2 Azione

La rete produce $a \in [-1, 1]$. La conversione nel comando del banco è

$$c = c_{min} + (1 - c_{min}) \frac{a + 1}{2} \in [c_{min}, 1], \quad u = c u_{max}, \quad c_{min} = 0.05, u_{max} = 20. \quad (9)$$

Quindi $u \in [1, 20]$: il minimo è sopra la soglia di arresto (0.5), così la rete non può spegnere il motore (una ripartenza costerebbe 200 ms), e il massimo è il punto in cui la mappa satura.

6.3 Osservazione

#	Grandezza	Perché
0	$F_{lc}/2.5$	forza misurata; 2.5 N $\approx$ spinta massima, così il valore sta circa in $[0, 1]$
1	e/F_{ref} , con $e = F_{ref} - F_{lc}$	errore normalizzato: la quantità da annullare
2	$\widehat{\Delta F}/2.5$	variazione della forza filtrata: informa sulla tendenza
3-5	$c_{k-1}, c_{k-2}, c_{k-3}$	ultimi tre comandi: con il ritardo di attuazione fanno parte dello stato
6	I/I_{max}	integrale dell'errore: permette di eliminare l'errore a regime

```
219 def _get_obs(self, force, delta_force, error):
220     obs = np.array(
221         [
222             force / self.f_scale,
223             error / self.force_ref,
224             delta_force / self.f_scale,
225             self.storia_c[-1],           # ultimo comando
226             self.storia_c[-2],         # penultimo
227             self.storia_c[-3],         # terzultimo
228             self.integral / self.i_max,
229         ],
230         dtype=np.float32,
231     )
232     return np.clip(obs, self.observation_space.low, self.observation_space.high)
```

Filtro della derivata. La differenza tra due misure consecutive è dominata dal rumore. Si usa un filtro esponenziale (EMA):

$$\widehat{\Delta F}_k = \widehat{\Delta F}_{k-1} + \beta((F_k - F_{k-1}) - \widehat{\Delta F}_{k-1}), \quad \beta = 0.2, \quad (10)$$

equivalente a un passa-basso del primo ordine con costante di tempo $\tau_{EMA} = -\Delta t / \ln(1 - \beta) = 0.02/0.223 \approx 90$ ms. Per un ingresso a rumore bianco la varianza in uscita è ridotta del fattore $\beta/(2 - \beta) = 0.11$.

Integrale con anti-windup.

$$I_k = \text{clip}(I_{k-1} + e_k \Delta t, -I_{max}, I_{max}), \quad I_{max} = 0.5 \text{ N s}. \quad (11)$$

La saturazione impedisce che, durante una saturazione del motore, l'integrale cresca senza limite. Il valore di I_{max} fissa anche la *sensibilità* dell'osservazione: con un errore di 0.1 N, I/I_{max} cresce di $0.1/0.5 = 0.2$ al secondo. Con il valore iniziale $I_{max} = 2$ cresceva di appena 0.05 al secondo, un segnale troppo debole perché la rete lo usasse (esperimento E).

Storia dei comandi e proprietà di Markov. Il RL presuppone che l'osservazione contenga tutto ciò che serve a prevedere il futuro (proprietà di Markov). Con un ritardo di n passi, i comandi $c_{k-1}, \dots, c_{k-n}$ sono già stati inviati ma non hanno ancora avuto effetto sulla forza: senza vederli la rete non sa che una correzione è “in viaggio” e ne aggiunge un'altra, innescando un'oscillazione. Tre comandi coprono il ritardo nominale.

6.4 Ricompensa

$$r = -e_n^2 - w_{e1} |e_n| - w_\Delta (\Delta c)^2 - w_u c^2, \quad e_n = \frac{e}{F_{ref}}, \quad w_{e1} = 1, w_\Delta = 5, w_u = 0.01. \quad (12)$$

- e_n^2 : penalizza molto gli errori grandi. Vicino a zero però è quasi piatto (derivata $2e_n \rightarrow 0$).
- $|e_n|$: derivata costante. Con un errore persistente di 0.15 N, $e_n = 0.130$: il termine quadratico costa 0.017 a passo, quello lineare 0.130, circa 8 volte di più. Senza il termine lineare la policy lasciava errori di questo tipo (esperimento A).
- $(\Delta c)^2$: penalizza un comando che cambia molto da un passo all'altro (usura, rumore, vibrazioni).
- c^2 : piccola penalità sullo sforzo, rompe la simmetria a favore di comandi bassi a parità di errore.

Normalizzare l'errore su F_{ref} rende i pesi indipendenti dalle unità di misura.

7 L'algoritmo SAC

7.1 Obiettivo

Nel RL classico si cerca la policy π che massimizza il ritorno scontato atteso $\mathbb{E}[\sum_k \gamma^k r_k]$. Il fattore γ fissa l'orizzonte efficace,

$$H \approx \frac{1}{1-\gamma} = 100 \text{ passi} = 2 \text{ s} \quad (\gamma = 0.99), \quad (13)$$

scelto perché l'errore lasciato da un cambio di inclinazione dura secondi: con $\gamma = 0.98$ ($H = 1$ s) il beneficio di correggerlo era poco visibile alla policy.

SAC (*Soft Actor-Critic*) massimizza un obiettivo che premia anche l'entropia della policy:

$$J(\pi) = \mathbb{E} \left[\sum_k \gamma^k (r_k + \alpha \mathcal{H}(\pi(\cdot | s_k))) \right], \quad \mathcal{H}(\pi(\cdot | s)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s)]. \quad (14)$$

Il termine di entropia, pesato dalla temperatura α , spinge a esplorare e impedisce che la policy diventi deterministica troppo presto.

7.2 Critic: la Q “soft”

Il critic stima $Q(s, a)$, il ritorno atteso partendo da s con l'azione a e poi seguendo π . Per l'obiettivo con entropia vale l'equazione di Bellman “soft”:

$$Q(s, a) = r + \gamma \mathbb{E}_{s', a' \sim \pi} [Q(s', a') - \alpha \log \pi(a' | s')]. \quad (15)$$

Si usano due reti critic Q_{ϕ_1}, Q_{ϕ_2} e, per il bersaglio, il minimo delle due versioni “lente” (*target network*) $Q_{\tilde{\phi}_j}$, per ridurre la sovrastima sistematica di Q :

$$y = r + \gamma \left(\min_{j=1,2} Q_{\tilde{\phi}_j}(s', a') - \alpha \log \pi(a' | s') \right), \quad a' \sim \pi(\cdot | s'), \quad \mathcal{L}_Q = \mathbb{E} [(Q_{\phi_j}(s, a) - y)^2]. \quad (16)$$

Le reti target seguono quelle principali con una media mobile: $\bar{\phi} \leftarrow \tau_p \phi + (1 - \tau_p) \bar{\phi}$, $\tau_p = 0.005$.

7.3 Actor e trucco della riparametrizzazione

L'actor produce media $\mu_\theta(s)$ e deviazione standard $\sigma_\theta(s)$ di una gaussiana, poi schiaccia con $\tanh$ per stare in $[-1, 1]$. Per poter derivare rispetto a θ attraverso il campionamento si scrive l'azione come funzione deterministica di un rumore esterno:

$$a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \varepsilon), \quad \varepsilon \sim \mathcal{N}(0, 1). \quad (17)$$

L'actor minimizza

$$\mathcal{L}_\pi = \mathbb{E}_{s, \varepsilon} [\alpha \log \pi_\theta(a|s) - \min_j Q_{\phi_j}(s, a)], \quad (18)$$

cioè cerca azioni con Q alto mantenendo la distribuzione larga.

Correzione della tanh. Se $z \sim \mathcal{N}(\mu, \sigma^2)$ e $a = \tanh z$, per il cambio di variabile di una densità

$$\pi(a|s) = \mathcal{N}(z; \mu, \sigma^2) \left| \frac{da}{dz} \right|^{-1} \Rightarrow \log \pi(a|s) = \log \mathcal{N}(z; \mu, \sigma^2) - \log(1 - \tanh^2 z). \quad (19)$$

Senza questo termine l'entropia sarebbe calcolata sulla gaussiana e non sull'azione vera, e la policy verrebbe premiata per spingere z verso valori enormi (azioni saturate a ± 1).

7.4 Temperatura automatica

Invece di fissare α a mano, SAC la regola per mantenere l'entropia attorno a un valore obiettivo $\bar{\mathcal{H}}$ (in Stable-Baselines3: $-\dim(\mathcal{A}) = -1$):

$$\mathcal{L}_\alpha = \mathbb{E}_{a \sim \pi} [-\alpha (\log \pi(a|s) + \bar{\mathcal{H}})]. \quad (20)$$

Se l'entropia è sotto l'obiettivo, $-\log \pi < \bar{\mathcal{H}}$, il gradiente fa crescere α (più premio all'esplorazione), e viceversa: è un anello di controllo sull'entropia.

7.5 Off-policy e replay buffer

SAC è *off-policy*: ogni transizione (s, a, r, s') viene salvata in un buffer (200 000 elementi) e le reti vengono aggiornate su minibatch estratti a caso dal buffer, anche se raccolti con versioni precedenti della policy. Questo rende SAC molto più efficiente nei campioni dei metodi on-policy.

7.6 Configurazione usata

```

357 model = SAC(
358     "MlpPolicy",
359     train_env,
360     learning_rate=3e-4,
361     buffer_size=200_000,
362     batch_size=256,
363     gamma=0.99,      # orizzonte  $\sim 1/(1-\text{gamma}) = 100$  passi = 2 s (era 1 s): l'errore che
                        # resta
364     "                # dopo un cambio di inclinazione dura secondi, la policy deve "vederlo"
365     verbose=1,
366     seed=0,
367 )
368 model.learn(total_timesteps=300_000, callback=eval_cb) # la v2 a 150k non era a
                        # convergenza
369 model.save("sac_fan")

```

Iperparametro	Valore	Origine
learning rate	$3 \cdot 10^{-4}$	impostato
buffer	200 000	impostato
batch	256	impostato
γ	0.99	impostato
passi di training	300 000	impostato
reti actor e critic	2×256 , ReLU	default SB3
numero di critic	2	default SB3
τ_p (target)	0.005	default SB3
α	automatica, $\bar{\mathcal{H}} = -1$	default SB3

Durante il training un `EvalCallback` valuta la policy ogni 5000 passi su 5 episodi *nominali* con azioni deterministiche, e salva il modello migliore in `best_sac_fan/best_model.zip`.

8 Esperimenti

8.1 Metriche

Il reward cambia da un esperimento all'altro, quindi non è confrontabile: si confrontano metriche fisiche, su 20 episodi randomizzati con semi fissi (gli stessi per tutte le policy), escludendo i primi 2 s (avvio):

- **RMS** dell'errore di forza;
- **offset lento**: valore medio assoluto dell'errore filtrato con un filtro mediano di 1 s (l'errore che resta dopo un cambio di inclinazione);
- **oscillazione**: deviazione standard dell'errore meno la sua componente lenta;
- $|\overline{\Delta c}|$: variazione media del comando per passo;
- spegnimenti del motore per episodio.

8.2 Risultati

Esp.	Modifica	RMS [N]	Offset [N]	Oscill. [N]	$ \overline{\Delta c} $	Spegn.
A	modello dalla sysid, reward e_n^2 , γ 0.98, 150k passi	0.136	0.106	0.057	0.012	0
D	+ termine $ e_n $, γ 0.99, 300k passi	0.117	0.085	0.061	0.021	0.25
E	+ I_{max} $2 \rightarrow 0.5$, $c_{min} = 0.05$	0.090	0.049	0.059	0.016	0

In condizioni nominali: A 0.127 N, D 0.108 N, E 0.078 N.

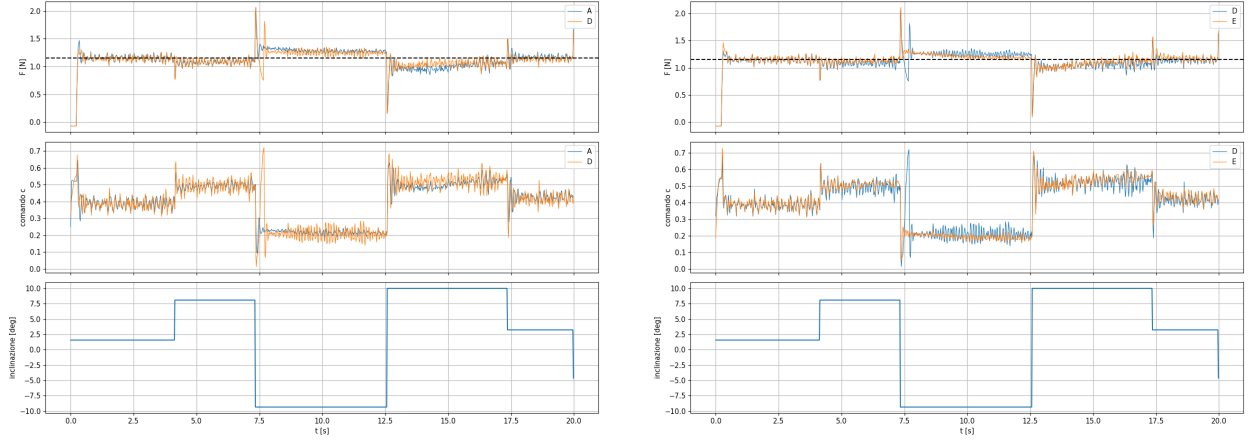


Figura 3: Stesso episodio randomizzato: A contro D (sinistra) e D contro E (destra).

A $\rightarrow$ D (reward shaping). Il termine lineare riduce l'offset del 20%, ma rende relativamente più conveniente muovere il comando: $|\Delta c|$ cresce del 75% e compaiono spegnimenti del motore (la policy portava il comando quasi a zero per correggere un overshoot).

D $\rightarrow$ E (scala delle osservazioni e vincolo sull'azione). Rendere l'integrale 4 volte più sensibile dimezza l'offset senza cambiare rete né reward; c_{min} elimina gli spegnimenti. È stato il miglioramento più grande.

8.3 Sul banco: la policy E

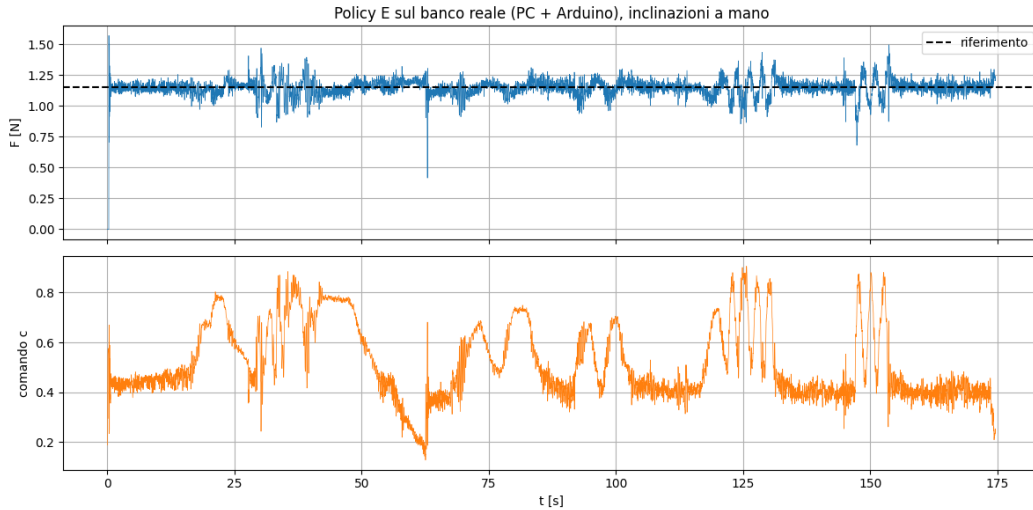


Figura 4: Policy E sul banco reale (PC + Arduino), 175 s con inclinazioni a mano.

RMS 0.067 N sulla prova da 175 s (0.022–0.030 N nei tratti a corsia ferma), errore medio +0.003 N: prestazioni nell'intervallo previsto dalla simulazione. Però la FFT dell'errore mostrava un picco a ≈ 3 Hz (fino al 47% della varianza tra 2 e 5 Hz in una prova breve), presente anche nel comando: un'oscillazione d'anello. In simulazione la policy E riproduce lo stesso spettro con un ritardo di 3 passi, e diventa quasi instabile con 4.

8.4 Esperimento F: robustezza al ritardo

Modifiche: ritardo randomizzato in $\{1, \dots, 4\}$ passi e ultimi tre comandi nell'osservazione. Valutazione con il ritardo forzato a ogni valore (12 episodi randomizzati per caso):

Ritardo	RMS [N]		Quota 2-5 Hz		$\overline{ \Delta c }$	
	E	F	E	F	E	F
1 passo (20 ms)	0.089	0.096	11%	8%	0.016	0.013
2 passi (40 ms)	0.101	0.105	20%	12%	0.020	0.014
3 passi (60 ms)	0.151	0.111	56%	20%	0.032	0.014
4 passi (80 ms)	0.237	0.118	76%	26%	0.045	0.014

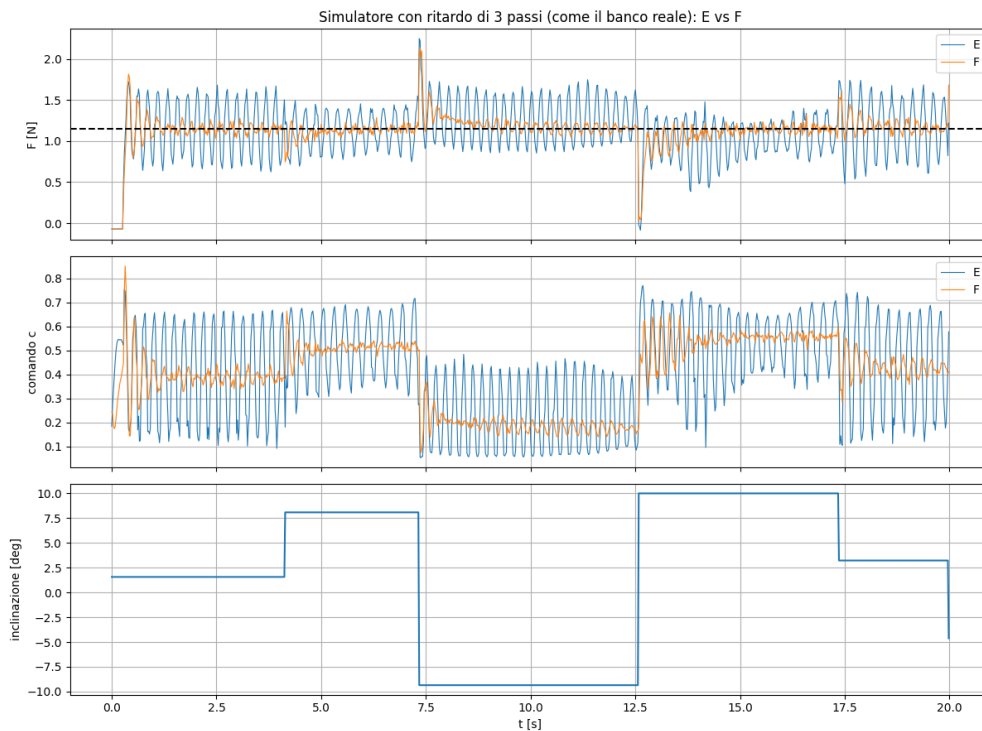


Figura 5: E contro F nello stesso episodio con ritardo di 3 passi.

Con il ritardo del banco F ha il 26% di errore in meno e circa un terzo dell'oscillazione; al crescere del ritardo peggiora poco, mentre E si avvicina all'instabilità. Il prezzo è una leggera perdita con ritardo piccolo: il compromesso tra robustezza e prestazioni nominali. Il miglior modello F è stato salvato a 110 000 passi.

Policy F sul banco La policy F è stata provata sul banco con PC + Arduino con esito positivo (osservazione diretta durante la prova). Il file della prova non è stato archiviato nel repository: il confronto quantitativo con E sul banco resta da fare con il protocollo di docs/05_prove_sul_banco.md.

9 Riepilogo delle lezioni

- Il simulatore va costruito sui dati del banco e validato confrontandolo con il banco a parità di comando, prima di addestrare.

- Il dimensionamento del compito (riferimento, angolo massimo) va fatto sui limiti fisici misurati, altrimenti la policy impara solo a saturare.
- Discretizzare la dinamica in modo esatto: è semplice e vale per ogni costante di tempo.
- La randomizzazione deve contenere la realtà con margine, in particolare sul ritardo.
- Con un ritardo, l'osservazione deve contenere i comandi non ancora arrivati (proprietà di Markov).
- La scala delle osservazioni conta quanto il reward: un segnale che varia poco è quasi invisibile alla rete.
- Vincolare lo spazio delle azioni (qui c_{min}) può eliminare comportamenti dannosi senza toccare il reward.
- Confrontare policy con metriche fisiche e semi fissi, mai con il reward se il reward è cambiato.

A Codice: parametri e costruttore dell'ambiente

```

41 PARAMETRI_DEFAULT = dict(
42     u_tab_moto=[1, 2, 3, 4, 6, 8, 10, 12, 15, 18, 20],
43     F_tab_moto=[0.123, 0.302, 0.456, 0.617, 0.873, 1.090, 1.331, 1.557, 1.835, 2.206, 2.206],
44     u_avvio=1.5, u_arresto=0.5,
45     tau_moto=0.078, tau_moto_min=0.033, tau_moto_max=0.112,
46     tau_arresto=0.174, ritardo_avvio=0.200, ritardo_moto=0.027,
47     rumore_s0=0.0011, rumore_s1=0.0208, zero_dopo=0.132,
48 )
49
50
51 def carica_parametri(percorso=None):
52     """Legge i parametri misurati; se il file non esiste usa i valori di riserva."""
53     percorso = Path(percorso) if percorso else Path(__file__).parent / "sysid_params.json"
54     par = dict(PARAMETRI_DEFAULT)
55     if percorso.exists():
56         with open(percorso) as f:
57             par.update(json.load(f))
58     return par
59
60
61 def __init__(
62     self,
63     force_ref=1.15,          # [N]    forza desiderata sulla load cell (centro della zona
64                             #        utile)
65     dt=0.02,                # [s]    periodo di controllo (50 Hz)
66     max_episode_steps=1000, # [-]    durata episodio (20 s)
67     domain_randomization=True,
68     u_max=20.0,             # [-]    comando massimo nella scala del banco (0..40)
69     c_min=0.05,            # [-]    comando minimo normalizzato: u_min = c_min*u_max =
70     1,                      #
71                             #        sopra la soglia di arresto -> la rete non puo'
72                             #        spegnere il motore
73     max_angle=14.0,         # [deg]  inclinazione massima della corsia
74     file_parametri=None,    # percorso di sysid_params.json (default: accanto a questo
75                             #        file)
76 ):
77     super().__init__()
78
79     # ----- Controllo -----
80     self.force_ref = force_ref
81     self.dt = dt
82     self.max_episode_steps = max_episode_steps
83     self.domain_randomization = domain_randomization
84     self.u_max = u_max

```

```

85     self.c_min = c_min
86     self.max_angle = max_angle
87
88     # ----- Parametri misurati -----
89     p = carica_parametri(file_parametri)
90     # aggiungo il punto (0, 0): sotto il primo livello misurato la spinta tende a zero
91     self.u_tab = np.r_[0.0, p["u_tab_moto"]]
92     self.F_tab = np.r_[0.0, p["F_tab_moto"]]
93     self.u_avvio_nom = p["u_avvio"]
94     self.u_arresto_nom = p["u_arresto"]
95     self.tau_nom = p["tau_moto"]
96     self.tau_min, self.tau_max = p["tau_moto_min"], p["tau_moto_max"]
97     self.tau_off_nom = p["tau_arresto"]
98     self.rit_avvio_nom = p["ritardo_avvio"]
99     self.rumore_s0 = p["rumore_s0"]
100    self.rumore_s1_nom = p["rumore_s1"]
101    self.zero_dopo = p["zero_dopo"]
102
103    self.g = 9.81
104    self.mass_nom = 0.317      # [kg] massa carrellino
105
106    # ----- Normalizzazioni / reward -----
107    self.f_scale = 2.5         # [N] scala per forza e derivata (spinta massima ~2.2 N)
108    # [N*s] saturazione integrale (anti-windup). Era 2.0: con un errore di 0.1 N l'
osservazione
109    # integral/i_max cresceva di solo 0.05 al secondo, un segnale troppo debole perche' la
rete
110    # lo usasse per togliere l'offset. Con 0.5 lo stesso errore la sposta 4 volte piu' in
fretta.
111    self.i_max = 0.5
112    self.w_e1 = 1.0           # peso del termine LINEARE sull'errore |e_n| (vedi reward
in step)
113    self.w_du = 5.0           # peso penalita' variazione comando
114    self.w_u = 0.01           # peso sullo sforzo di controllo
115    self.beta_df = 0.2        # coefficiente EMA per filtrare dF_lc
116
117    # ----- Spazi -----
118    self.action_space = spaces.Box(low=-1.0, high=1.0, shape=(1,), dtype=np.float32)
119    obs_high = np.array([10.0, 10.0, 10.0, 1.0, 1.0, 1.0, 1.0], dtype=np.float32)
120    obs_low = np.array([-10.0, -10.0, -10.0, 0.0, 0.0, 0.0, -1.0], dtype=np.float32)
121    self.observation_space = spaces.Box(low=obs_low, high=obs_high, dtype=np.float32)

```

B Codice: reset dell'episodio

```

238    def reset(self, seed=None, options=None):
239        super().reset(seed=seed)
240        self._sample_parameters()
241
242        self.thrust = 0.0
243        self.in_moto = False
244        self.conta_avvio = 0
245        self.coda_u = deque([0.0] * self.n_ritardo)    # comandi "in viaggio" verso l'ESC
246        self.last_c = 0.0
247        self.storia_c = deque([0.0, 0.0, 0.0], maxlen=3)    # ultimi 3 comandi, il piu' recente
a destra
248        self.integral = 0.0
249        self.df_filt = 0.0
250        self.t = 0
251
252        self.angle = self.np_random.uniform(-self.max_angle, self.max_angle)
253        self._schedule_disturbance()
254        self.angle_target = self.angle
255

```

```
256         force = self._measure()
257         error = self.force_ref - force
258         self.last_force = force
259
260         obs = self._get_obs(force, 0.0, error)
261         info = {"force": force, "force_ref": self.force_ref, "angle": self.angle, "ESC": 0.0, "
u": 0.0}
262         return obs, info
```